

Tensor Train

Deep Understanding Guide for Questions 1–9

This document expands the section **Tensor Train** from the uploaded exam guide. It assumes no prior tensor-train background, develops each question from zero, and follows the reference map listed in the guide.

Prepared as a single PDF study document

April 26, 2026

Contents

How to Use This Guide	2
Zero-Knowledge Warm-Up: Why Tensor Train Exists	2
1 Existence of the Tensor Train	4
2 Unfolding Matrices for the Tensor Train	6
3 Minimal TT and the Connection Between TT Ranks and Unfolding Ranks	8
4 Arithmetic in Tensor-Train Format	10
5 Orthogonality and Orthogonalization in Tensor-Train Format	13
6 TT-SVD, Quasi-Optimal TT Recompression, and Error Estimate	15
7 Idea of the TT-Cross Method and TT-Cross Without Adaptive Rank Search	17
8 Adaptive Rank Search in TT: Ideas, Problems, and Solutions	20
9 Fast Elementwise Operations on TT Tensors Through a Special Version of TT-Cross	22
One-Page Exam Summary	24
Bibliography from the Uploaded Guide	24

How to Use This Guide

The Tensor Train section has nine questions. They form a progression from the representation itself to algorithms that compute, compress, and use it:

1. What a Tensor Train is and why an exact TT representation always exists.
2. Which unfolding matrices are natural for TT and why they differ from ordinary mode unfoldings.
3. Why minimal TT ranks are exactly the ranks of these unfoldings.
4. How arithmetic works in TT format and why rounding is essential.
5. How orthogonalization makes TT computations stable.
6. How TT-SVD and TT-rounding produce quasi-optimal low-rank approximations with error control.
7. How TT-cross constructs a TT approximation from sampled entries when ranks are fixed.
8. How adaptive rank search works when the ranks are unknown.
9. How special TT-cross variants accelerate elementwise and nonlinear operations.

Main mental model. A TT tensor is a chain. Each dimension contributes one small core, and neighboring cores communicate through hidden indices called TT ranks. If these hidden links stay small, a huge tensor can be stored and manipulated using a number of parameters proportional to the number of dimensions.

The attached guide points especially to Oseledets for the TT definition, existence theorem, rounding, and arithmetic; Oseledets–Tyrtysnikov for TT-cross; Holtz–Rohwedder–Schneider for ALS/MALS optimization in TT; Dolgov–Savostyanov for AMEn and rank adaptivity; Savostyanov for maxvol-style tensor cross interpolation; Kolda–Bader for general tensor notation; and Trefethen–Bau for matrix SVD foundations.

Zero-Knowledge Warm-Up: Why Tensor Train Exists

A vector has one index, a matrix has two indices, and a tensor has several indices:

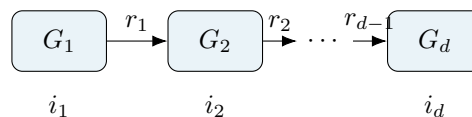
$$\mathbf{x} = (x_i), \quad \mathbf{A} = (a_{ij}), \quad \mathcal{X} = (x_{i_1 i_2 \dots i_d}).$$

The number of indices is called the **order** or the number of **modes**. A tensor of size $n \times n \times \dots \times n$ with d modes has n^d entries. Even when n is modest, this number quickly becomes impossible to store.

Tensor decompositions try to exploit hidden structure. Instead of storing every entry, they store small building blocks from which entries can be reconstructed. Tensor Train, abbreviated TT, represents one tensor entry by multiplying a chain of small matrices:

$$x_{i_1 \dots i_d} = \mathbf{G}_1(i_1) \mathbf{G}_2(i_2) \cdots \mathbf{G}_d(i_d).$$

The object $\mathbf{G}_k(i_k)$ is a matrix slice chosen from the k -th core. The first slice is a row vector and the last slice is a column vector, so the final product is a scalar.



The hidden link sizes $r_1, \dots, r_{d-1}$ are the **TT ranks**. They play a role similar to matrix rank: small TT ranks mean the tensor has compressible structure.

A useful analogy. A matrix factorization writes a large table as left factors, a small interaction, and right factors. TT repeats this idea many times along a chain: each split between left indices and right indices has its own rank.

Important notation

Symbol	Meaning
$\mathcal{X}$	Original d -dimensional tensor.
i_k	Index in mode k , with $1 \leq i_k \leq n_k$.
$\mathcal{G}_k$	The k -th TT core, a three-dimensional array of size $r_{k-1} \times n_k \times r_k$.
$\mathbf{G}_k(i_k)$	Matrix slice of core k , of size $r_{k-1} \times r_k$.
r_k	TT rank between cores k and $k+1$. Boundary ranks are $r_0 = r_d = 1$.
$\mathbf{X}^{(k)}$	TT unfolding that groups $(i_1, \dots, i_k)$ as rows and $(i_{k+1}, \dots, i_d)$ as columns.
$\ \mathcal{X}\ _F$	Frobenius norm, the square root of the sum of squared entries.

1 Existence of the Tensor Train

Question in plain language

What is the TT format, and why can every finite tensor be written exactly as a TT tensor?

Start from zero

For a high-dimensional tensor, one entry has many indices:

$$x_{i_1, i_2, \dots, i_d}.$$

The TT format writes this scalar as a product of small matrices selected by the indices:

$$x_{i_1, i_2, \dots, i_d} = \mathbf{G}_1(i_1) \mathbf{G}_2(i_2) \cdots \mathbf{G}_d(i_d).$$

Here each $\mathbf{G}_k(i_k)$ is a matrix slice. Because $r_0 = r_d = 1$, the first slice has size $1 \times r_1$, the last slice has size $r_{d-1} \times 1$, and the product is a 1×1 scalar.

Equivalently, the full core $\mathcal{G}_k$ has size

$$r_{k-1} \times n_k \times r_k.$$

The physical index i_k chooses which slice to use, while the hidden indices of sizes r_{k-1} and r_k connect neighboring cores.

Core intuition. A TT core is like a local translator: it receives a hidden state from the left, reads the current physical index, and sends a hidden state to the right. The final scalar is obtained after passing through all modes.

Storage cost

The total number of stored parameters is

$$\sum_{k=1}^d r_{k-1} n_k r_k.$$

If all mode sizes are approximately n and all TT ranks are approximately r , then the storage is about

$$dnr^2.$$

This is the main reason TT is useful: full storage is n^d , but TT storage is linear in d when ranks remain moderate.

Why an exact TT representation always exists

The constructive proof is based on repeated matrix SVDs.

Start with the full tensor $\mathcal{X}$. Reshape it into a matrix that separates the first index from all remaining indices:

$$\mathbf{X}^{(1)} \in \mathbb{R}^{n_1 \times (n_2 \cdots n_d)}.$$

Take its SVD and split it into a left factor and a remaining right factor. The left factor becomes the first TT core. Then reshape the remaining factor so that the second index is separated from the rest, take another SVD, and continue.

At step k , the current matrix is split into a left part involving $(i_1, \dots, i_k)$ and a right part involving $(i_{k+1}, \dots, i_d)$. The rank of that split becomes the TT rank r_k .

Existence theorem in words. Repeated exact SVDs turn any finite tensor into a chain of TT cores. Therefore, unlike CP with a prescribed rank, the TT representation itself does not suffer from non-existence.

A small three-dimensional example

For $d = 3$, the TT formula is

$$x_{ijk} = \mathbf{G}_1(i) \mathbf{G}_2(j) \mathbf{G}_3(k).$$

The core shapes are

$$\mathcal{G}_1 : 1 \times n_1 \times r_1, \quad \mathcal{G}_2 : r_1 \times n_2 \times r_2, \quad \mathcal{G}_3 : r_2 \times n_3 \times 1.$$

This is closely related to matrix factorizations of unfoldings, but the representation is arranged as a chain rather than as one central core.

What to reproduce in an exam

- State the TT entry formula as a product of core slices.
- State the core sizes $r_{k-1} \times n_k \times r_k$ and boundary ranks $r_0 = r_d = 1$.
- Explain the storage cost $\sum_k r_{k-1} n_k r_k$, and why it is linear in d for bounded ranks.
- Explain exact existence by sequential SVDs of unfolding matrices.

Common pitfalls

- Do not say TT rank is one number. It is a vector $(r_1, \dots, r_{d-1})$.
- Do not confuse the physical dimensions n_k with the TT ranks r_k . The former are tensor mode sizes; the latter are hidden link sizes.
- Do not claim TT factors are unique. There is gauge freedom between neighboring cores.

Reading map from the uploaded guide

- [KB09, pp. 455–464] for tensor notation and unfoldings.
- [O11, pp. 2295–2298] for the TT definition and motivation.
- [O11, pp. 2298–2301] for exact existence via unfoldings and sequential SVD.
- [TB97, pp. 25–45] for SVD as the matrix foundation.

2 Unfolding Matrices for the Tensor Train

Question in plain language

Which matrices are formed from a tensor in TT theory, and how do they reveal the TT ranks?

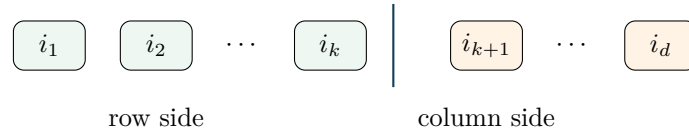
Start from zero: what is an unfolding?

An unfolding is a reshaping of a tensor into a matrix. No entries are changed; only the indexing is reorganized. For TT, the most important unfolding is not the usual mode- n unfolding. Instead, TT uses **left-right separation unfoldings**.

For each split position $k = 1, \dots, d - 1$, define

$$\mathbf{X}^{(k)} \in \mathbb{R}^{(n_1 \cdots n_k) \times (n_{k+1} \cdots n_d)}.$$

The rows are indexed by the left multi-index $(i_1, \dots, i_k)$, and the columns are indexed by the right multi-index $(i_{k+1}, \dots, i_d)$.



There are exactly $d - 1$ such unfoldings, one for each cut between neighboring modes.

Why these unfoldings match the TT chain

In a TT tensor, the cut after mode k separates the left cores from the right cores:

$$\mathbf{G}_1(i_1) \cdots \mathbf{G}_k(i_k) \quad \text{and} \quad \mathbf{G}_{k+1}(i_{k+1}) \cdots \mathbf{G}_d(i_d).$$

The only connection between the two sides is the hidden index of size r_k . Therefore the matrix $\mathbf{X}^{(k)}$ factors through a dimension r_k . This implies

$$\text{rank}(\mathbf{X}^{(k)}) \leq r_k.$$

For a minimal TT representation, this inequality becomes equality.

Relation to ordinary mode unfoldings

A mode unfolding isolates one mode from all other modes. For example, a mode-2 unfolding has rows indexed by i_2 and columns indexed by all other indices. TT unfoldings isolate a **contiguous block** of modes from its complement:

$$(i_1, \dots, i_k) \quad \text{versus} \quad (i_{k+1}, \dots, i_d).$$

This difference matters because the TT format is a chain. Its ranks measure separations along the chain, not individual mode ranks.

Interpretation. The rank of $\mathbf{X}^{(k)}$ measures how much information must cross the boundary between modes k and $k + 1$. The TT rank r_k is exactly the size of that communication channel in a minimal representation.

How these unfoldings are used in TT-SVD

TT-SVD repeatedly forms or implicitly manipulates these left-right unfoldings. At each step, it computes a truncated SVD of the current unfolding. The retained rank becomes the next TT rank, and the remaining factor is reshaped for the next step.

This is why the unfoldings are not just definitions. They are the actual matrices whose ranks and singular values control TT construction and TT approximation.

What to reproduce in an exam

- Define $\mathbf{X}^{(k)}$ and give its size.
- Explain that there are $d - 1$ TT unfoldings.
- Explain that their ranks bound and, in minimal TT form, equal TT ranks.
- Contrast them with ordinary mode unfoldings.

Common pitfalls

- Do not define TT ranks using only mode unfoldings. TT ranks come from left-right block unfoldings.
- Do not forget that the left block is contiguous in the chosen mode ordering.
- Do not treat the ordering of modes as irrelevant. A different ordering can change TT ranks.

Reading map from the uploaded guide

- [O11, pp. 2297–2299] for TT unfolding matrices and rank definitions.
- [O11, pp. 2299–2301] for the constructive proof by sequential decompositions.
- [KB09, pp. 459–464] for general matricization background.
- [TB97, pp. 25–45] for matrix rank and SVD foundations.

3 Minimal TT and the Connection Between TT Ranks and Unfolding Ranks

Question in plain language

What is a minimal TT representation, and why are its ranks equal to the ranks of the TT unfoldings?

Minimal TT ranks

The minimal TT ranks of $\mathcal{X}$ are

$$r_k = \text{rank}(\mathbf{X}^{(k)}), \quad k = 1, \dots, d-1.$$

This is the central rank theorem for TT.

The theorem has two directions:

- If a TT representation has rank r_k , then the unfolding rank after mode k cannot exceed r_k .
- Sequential exact SVD constructs a TT representation whose rank after mode k is exactly $\text{rank}(\mathbf{X}^{(k)})$.

Together, these show that the minimal possible rank is the unfolding rank.

Why the theorem is true

Fix a split k . In TT form,

$$x_{i_1 \dots i_d} = \mathbf{G}_1(i_1) \cdots \mathbf{G}_k(i_k) \mathbf{G}_{k+1}(i_{k+1}) \cdots \mathbf{G}_d(i_d).$$

Introduce a hidden index $\alpha_k = 1, \dots, r_k$ at the cut. Then the entry can be written as

$$x_{i_1 \dots i_d} = \sum_{\alpha_k=1}^{r_k} L_{(i_1 \dots i_k), \alpha_k} R_{\alpha_k, (i_{k+1} \dots i_d)}.$$

In matrix form, this says

$$\mathbf{X}^{(k)} = \mathbf{L}_k \mathbf{R}_k.$$

Since the inner dimension is r_k , the rank of the unfolding is at most r_k . Exact SVD proves that this bound can be reached with equality.

Mental picture. Cut the TT chain at one link. Everything to the left becomes a matrix $\mathbf{L}_k$. Everything to the right becomes a matrix $\mathbf{R}_k$. The size of the cut link is the factorization dimension of the unfolding.

Gauge freedom and non-uniqueness

Even when ranks are minimal, TT cores are not unique. Between two neighboring cores one may insert an invertible matrix and its inverse:

$$\mathbf{G}_k(i_k) \mathbf{G}_{k+1}(i_{k+1}) = (\mathbf{G}_k(i_k) \mathbf{M})(\mathbf{M}^{-1} \mathbf{G}_{k+1}(i_{k+1})).$$

This changes the cores but not the represented tensor. This freedom is called **gauge freedom**. Orthogonalization chooses a convenient gauge.

Closedness of bounded TT-rank sets

For a fixed TT rank bound $(r_1, \dots, r_{d-1})$, the set of tensors satisfying

$$\text{rank}(\mathbf{X}^{(k)}) \leq r_k$$

for every k is closed, because it is described by matrix rank constraints on unfoldings. Matrix sets of bounded rank are closed.

This is a major difference from CP rank. Sets of tensors with CP rank at most R may fail to be closed, which can cause non-existence of best low-rank CP approximations. TT rank avoids this pathology because it is controlled by matrix ranks of unfoldings.

Important consequence. For fixed TT ranks, the best approximation problem is well posed: a best tensor of those ranks exists. The numerical algorithm may still be hard, but the target set itself is closed.

Ordering matters

The ranks r_k depend on which modes are placed next to each other in the TT chain. A tensor may have small TT ranks in one ordering and large ranks in another. This is why mode ordering is a practical modeling decision.

What to reproduce in an exam

- State the theorem $r_k = \text{rank}(\mathbf{X}^{(k)})$ for minimal TT ranks.
- Explain the factorization of the unfolding across a TT cut.
- Explain gauge freedom between neighboring cores.
- Explain why bounded TT-rank sets are closed and contrast this with CP rank.

Common pitfalls

- Minimal ranks are unique, but minimal cores are not unique.
- Closedness does not mean every algorithm will find the best approximation quickly.
- TT rank depends on mode ordering; it is not a completely order-free invariant.

Reading map from the uploaded guide

- [O11, pp. 2297–2301] for the theorem connecting TT ranks and unfolding ranks.
- [O11, pp. 2301–2306] for approximation, rounding, and stability of TT ranks.
- [DSL08, pp. 1084–1092] for contrast with CP rank and ill-posed low-rank approximation.
- [KB09, pp. 470–473] for tensor-rank pathologies used as comparison.

4 Arithmetic in Tensor-Train Format

Question in plain language

How do we add, multiply, and take inner products of TT tensors without expanding the full tensor?

Start from zero

A TT tensor is useful only if we can compute with it directly. If every operation required forming all n^d entries, the format would lose its purpose. TT arithmetic works locally with cores and then usually applies rounding to reduce ranks.

Assume two tensors $\mathcal{X}$ and $\mathcal{Y}$ are represented in TT form:

$$x_{i_1 \dots i_d} = \mathbf{G}_1(i_1) \cdots \mathbf{G}_d(i_d), \quad y_{i_1 \dots i_d} = \mathbf{H}_1(i_1) \cdots \mathbf{H}_d(i_d).$$

Addition

The sum $\mathcal{Z} = \mathcal{X} + \mathcal{Y}$ can be represented by block cores. For middle cores,

$$\mathbf{Z}_k(i_k) = \begin{bmatrix} \mathbf{G}_k(i_k) & 0 \\ 0 & \mathbf{H}_k(i_k) \end{bmatrix}.$$

At the first core, one concatenates horizontally; at the last core, one concatenates vertically. The TT ranks of the sum are at most the sums of the ranks:

$$r_k(\mathcal{X} + \mathcal{Y}) \leq r_k(\mathcal{X}) + r_k(\mathcal{Y}).$$

Addition is exact in this block construction, but ranks grow. Repeated additions can quickly produce large ranks, so TT-rounding is normally applied after arithmetic.

Scalar product

The scalar product is

$$\langle \mathcal{X}, \mathcal{Y} \rangle = \sum_{i_1, \dots, i_d} x_{i_1 \dots i_d} y_{i_1 \dots i_d}.$$

TT computes this by sequential contraction. Define a running small matrix $\mathbf{Z}_0 = 1$. Then update

$$\mathbf{Z}_k = \sum_{i_k=1}^{n_k} \mathbf{G}_k(i_k)^\top \mathbf{Z}_{k-1} \mathbf{H}_k(i_k), \quad k = 1, \dots, d.$$

The final value $\mathbf{Z}_d$ is a scalar. The important point is that one contracts core by core, never forming the full tensor.

Frobenius norm

The Frobenius norm follows from the scalar product:

$$\|\mathcal{X}\|_F = \sqrt{\langle \mathcal{X}, \mathcal{X} \rangle}.$$

If the TT is orthogonalized, norm computation can become even simpler, because many contractions collapse to identities.

Hadamard product

The elementwise product $\mathcal{Z} = \mathcal{X} \odot \mathcal{Y}$ is defined by

$$z_{i_1 \dots i_d} = x_{i_1 \dots i_d} y_{i_1 \dots i_d}.$$

It has TT slices

$$\mathbf{Z}_k(i_k) = \mathbf{G}_k(i_k) \otimes \mathbf{H}_k(i_k),$$

where $\otimes$ is the Kronecker product. The ranks multiply:

$$r_k(\mathcal{X} \odot \mathcal{Y}) \leq r_k(\mathcal{X}) r_k(\mathcal{Y}).$$

This can be much worse than addition, so rounding is again essential.

Applying separable operators

If a linear operator is itself in a tensor-product or TT-matrix format, applying it to a TT tensor can be done corewise. The resulting ranks generally multiply with the operator ranks. This is the basis for solving high-dimensional linear systems and eigenvalue problems in TT format.

Why recompression is part of arithmetic

Arithmetic tends to increase ranks:

Operation	Rank behavior	Usual follow-up
Addition	ranks add	round/compress
Hadamard product	ranks multiply	round/compress
Scalar product	no output ranks	sequential contraction
Linear operator application	ranks usually multiply	round/compress

TT-rounding converts the exact but high-rank result into a lower-rank approximation with controlled error.

What to reproduce in an exam

- Describe addition by block cores and rank addition.
- Describe the scalar product as sequential contraction of small matrices.
- Describe the Hadamard product by Kronecker products of slices and rank multiplication.
- Explain why ranks grow and why TT-rounding is normally applied after operations.

Common pitfalls

- Do not expand the full tensor when describing TT arithmetic.
- Do not ignore rank growth. Exact arithmetic in TT can still become expensive if ranks are not recompressed.
- Do not confuse Hadamard product rank growth with addition rank growth: addition adds ranks; Hadamard product multiplies them.

Reading map from the uploaded guide

- [O11, pp. 2308–2311] for addition, scalar product, Hadamard product, and contraction in TT.
- [O11, pp. 2302–2306] for rounding after rank growth.
- [HRS12, pp. A683–A695] for local tensor operations in TT optimization.
- [KB09, pp. 455–464] for tensor operation notation.

5 Orthogonality and Orthogonalization in Tensor-Train Format

Question in plain language

What does it mean for TT cores to be orthogonal, how do we orthogonalize them, and why is this useful?

Start from zero: orthogonality in matrices

For a matrix $\mathbf{Q}$, orthonormal columns mean

$$\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}.$$

This is valuable because orthogonal transformations preserve lengths and make least-squares problems stable.

TT orthogonality applies the same idea to reshaped cores. A TT core $\mathcal{G}_k$ has size $r_{k-1} \times n_k \times r_k$. It can be reshaped into a matrix in two useful ways.

Left-orthogonal cores

A core is left-orthogonal if the matrix obtained by merging r_{k-1} and n_k into the row index has orthonormal columns:

$$\sum_{i_k=1}^{n_k} \mathbf{G}_k(i_k)^\top \mathbf{G}_k(i_k) = \mathbf{I}_{r_k}.$$

This means the left part of the chain behaves like an isometry from the link space to the physical left block.

Right-orthogonal cores

A core is right-orthogonal if the matrix obtained by merging n_k and r_k into the column index has orthonormal rows:

$$\sum_{i_k=1}^{n_k} \mathbf{G}_k(i_k) \mathbf{G}_k(i_k)^\top = \mathbf{I}_{r_{k-1}}.$$

This means the right part of the chain behaves like an isometry.

How orthogonalization works

To make cores left-orthogonal, sweep from left to right:

1. Reshape $\mathcal{G}_k$ into a matrix of size $(r_{k-1}n_k) \times r_k$.
2. Compute a QR factorization $\mathbf{G}_k^{\text{mat}} = \mathbf{Q}\mathbf{R}$.
3. Replace the core by the reshaped $\mathbf{Q}$.
4. Multiply $\mathbf{R}$ into the next core.

The product represented by the chain remains unchanged. We only move the gauge from one core to the next.

To make cores right-orthogonal, sweep from right to left with the analogous QR or RQ factorization.

Mixed-canonical form

A TT can be left-orthogonal up to some core $p - 1$, right-orthogonal after core $p + 1$, and have one central non-orthogonal core p . This is called mixed-canonical form:

left-orthogonal cores | center core | right-orthogonal cores.

This form is the TT analogue of using orthonormal bases around a small active variable.

Why it matters. In mixed-canonical form, optimizing or truncating the center core is well conditioned because the surrounding left and right blocks act like orthonormal bases. Norms and local least-squares objectives become simpler.

Norm computation

If all cores except one are orthogonal, then the norm of the full tensor equals the norm of the center object. This is extremely useful for rounding and optimization. Instead of contracting the whole chain in an ill-conditioned way, one can move the orthogonality center and work locally.

Gauge transformations

Orthogonalization is a special case of gauge transformation. We can insert $\mathbf{M}\mathbf{M}^{-1}$ between neighboring cores without changing the tensor. Orthogonalization chooses $\mathbf{M}$ so that one core becomes orthogonal and the compensating factor is transferred to its neighbor.

What to reproduce in an exam

- Define left and right orthogonality through reshaped cores.
- Explain QR-based left-to-right and right-to-left sweeps.
- Explain mixed-canonical form and the orthogonality center.
- Explain why orthogonal form helps norms, truncation, and local optimization.

Common pitfalls

- Orthogonalizing a TT does not change the represented tensor; it changes only the gauge.
- Left-orthogonality and right-orthogonality use different core matricizations.
- Orthogonality is not the same as low rank. It improves numerical conditioning but does not by itself reduce ranks.

Reading map from the uploaded guide

- [O11, pp. 2302–2306] for orthogonalization and TT-rounding.
- [HRS12, pp. A683–A695] for ALS in TT and the role of canonical forms.
- [DS14, pp. A2248–A2258] for rank-adaptive TT/AMEn and basis enrichment.
- [GVL13, pp. 248–288] for QR machinery behind orthogonalization.

6 TT-SVD, Quasi-Optimal TT Recompression, and Error Estimate

Question in plain language

How does TT-SVD build a TT approximation, how is the error controlled, and why is the result called quasi-optimal?

The exact TT-SVD idea

TT-SVD is the constructive existence proof turned into an algorithm. Starting from the full tensor:

1. Reshape the tensor into a matrix separating the first mode from the rest.
2. Compute an SVD.
3. Keep the needed singular vectors and reshape them into the first core.
4. Multiply singular values into the remaining factor and continue with the next mode.

If no singular values are discarded, the result is an exact TT representation.

Truncated TT-SVD

For approximation, truncate each SVD. At split k , discard singular values whose tail norm is at most ε_k . The local discarded energy is

$$\left(\sum_{j>r_k} \sigma_j^2 \right)^{1/2} \leq \varepsilon_k.$$

The total error satisfies

$$\|\mathcal{X} - \mathcal{Y}\|_F \leq \left(\sum_{k=1}^{d-1} \varepsilon_k^2 \right)^{1/2}.$$

A common choice is

$$\varepsilon_k = \frac{\delta}{\sqrt{d-1}},$$

which guarantees $\|\mathcal{X} - \mathcal{Y}\|_F \leq \delta$.

Error intuition. Each truncated SVD creates a local error. Because the algorithm maintains orthogonality, these local errors add in squared norm, like perpendicular vectors.

Quasi-optimality

Let $\mathcal{X}_{\text{best}}$ be the best approximation with TT ranks bounded by $(r_1, \dots, r_{d-1})$. TT-SVD is not always exactly optimal, but it satisfies a quasi-optimal estimate of the form

$$\|\mathcal{X} - \mathcal{Y}\|_F \leq \sqrt{d-1} \|\mathcal{X} - \mathcal{X}_{\text{best}}\|_F.$$

The factor $\sqrt{d-1}$ is the price paid for solving a sequence of matrix truncation problems rather than the full nonlinear best TT approximation problem.

TT-rounding for a tensor already in TT format

Often the input is already a TT tensor with large ranks, perhaps after addition or Hadamard products. TT-rounding compresses it without forming the full tensor:

1. Orthogonalize the TT from right to left or left to right.
2. Sweep in the opposite direction.
3. At each link, reshape the core, compute a truncated SVD, and absorb the remaining factor into the next core.

The cost depends on mode sizes and TT ranks, not on the full number of entries.

Absolute and relative tolerances

If the desired relative error is η , set the absolute target to

$$\delta = \eta \|\mathcal{X}\|_F.$$

Then distribute δ across the $d - 1$ truncations. In practice, adaptive local choices may be used, but the equal distribution gives a clean exam-ready rule.

Cost intuition

For uniform mode size n and rank r , many TT-rounding operations scale roughly like

$$dnr^3$$

up to constants and details of neighboring ranks. This is far smaller than operations on the full tensor when r is moderate.

What to reproduce in an exam

- Describe exact TT-SVD as sequential SVD of left-right unfoldings.
- Explain truncation and local error tolerances.
- State the error bound based on the square root of the sum of squared local errors.
- State the quasi-optimal factor $\sqrt{d - 1}$.
- Explain TT-rounding for a tensor already in TT format.

Common pitfalls

- TT-SVD requires access to the full tensor or at least to large unfoldings; TT-cross is used when only entries are available.
- Quasi-optimal does not mean exactly optimal.
- The tolerance must be distributed across truncation steps, not applied independently as the full target at every step.

Reading map from the uploaded guide

- [O11, pp. 2298–2302] for exact and approximate TT-SVD with the error theorem.
- [O11, pp. 2302–2306] for TT-rounding in already compressed form.
- [TB97, pp. 25–45] for Eckart–Young and SVD background.
- [VVM12, pp. A1027–A1038] for an analogy with sequential Tucker truncation.

7 Idea of the TT-Cross Method and TT-Cross Without Adaptive Rank Search

Question in plain language

How can we build a TT approximation when we can evaluate individual tensor entries but cannot store or scan the full tensor?

Start from zero: why TT-SVD may be impossible

TT-SVD is powerful, but it needs access to large unfoldings or the full tensor. In many high-dimensional problems, a single entry can be computed cheaply, while the full tensor has too many entries to form. Examples include parameter-dependent functions, discretized high-dimensional PDE solutions, and elementwise functions of existing TT tensors.

TT-cross addresses this situation. It treats the tensor as a black box:

$$(i_1, \dots, i_d) \mapsto x_{i_1 \dots i_d}.$$

The algorithm chooses informative multi-indices and reconstructs a TT approximation from sampled entries.

Matrix cross approximation as the starting point

For a matrix, cross approximation selects rows I and columns J and forms

$$\mathbf{A} \approx \mathbf{A}_{:,J} \mathbf{A}_{I,J}^{-1} \mathbf{A}_{I,:}.$$

The intersection $\mathbf{A}_{I,J}$ should be well conditioned. Maxvol-style selection tries to choose a submatrix with large volume so the interpolation is stable.

TT-cross generalizes this idea to tensors. Instead of rows and columns, it chooses nested sets of left and right multi-indices.

Nested index sets

At split k , one needs representative left indices

$$(i_1, \dots, i_k)$$

and representative right indices

$$(i_{k+1}, \dots, i_d).$$

These sets must be nested along the chain so that each local core can be reconstructed consistently from sampled entries. The nestedness is what makes TT-cross more subtle than applying matrix cross independently to each unfolding.

Fixed-rank TT-cross

When the TT ranks are known in advance, the algorithm keeps index sets of fixed sizes r_k . A typical sweep does the following:

1. Fix right index sets and move from left to right.
2. At each core, evaluate a local block of tensor entries.

3. Use maxvol or a similar selection rule to update the left index set.
4. Then sweep from right to left and update right index sets.
5. Repeat until the selected cores or approximation stabilize.

The output is a TT tensor with the prescribed ranks.

Main intuition. TT-cross never tries to see the whole tensor. It asks selected questions: “What is the tensor value at this multi-index?” The algorithm chooses the questions so that the answers determine good local TT cores.

Role of maxvol

Maxvol chooses well-conditioned interpolation submatrices. In TT-cross, this improves the stability of the selected nested index sets. Poor index choices can lead to large interpolation coefficients and inaccurate approximations even if the nominal rank is correct.

TT-cross versus TT-SVD

Feature	TT-SVD	TT-cross
Input access	Full tensor or large unfoldings	Black-box entry evaluations
Rank handling	Truncates by singular values	Often needs prescribed or adaptively selected ranks
Main tool	SVD	Cross interpolation and maxvol
Best use case	Available dense tensor or existing TT rounding	Expensive high-dimensional black-box arrays

What to reproduce in an exam

- Explain black-box entry access.
- Recall matrix cross approximation as the analogy.
- Explain nested left and right multi-index sets.
- Describe fixed-rank sweeps and maxvol updates.
- Explain why TT-cross differs from TT-SVD.

Common pitfalls

- TT-cross is not based on singular values of full unfoldings.
- Fixed-rank TT-cross assumes ranks are known or chosen beforehand.
- Entry sampling must be structured; random samples alone do not automatically give TT cores.

Reading map from the uploaded guide

- [OT10, pp. 70–76] for TT-cross motivation and interpolation formula.
- [OT10, pp. 76–84] for fixed-rank sweeps and maxvol index updates.
- [GOSTZ10, pp. 247–256] for maxvol submatrix search used inside cross methods.
- [S14, pp. 217–230] for nested interpolation sets and quasi-optimality background.

8 Adaptive Rank Search in TT: Ideas, Problems, and Solutions

Question in plain language

What if the correct TT ranks are unknown, and how do adaptive methods increase ranks safely?

Why adaptivity is needed

In real problems, the ranks are usually not known. If ranks are too small, the approximation cannot represent the tensor accurately. If ranks are too large, computations become expensive and unstable. Adaptive rank search tries to find ranks automatically.

The challenge is that TT ranks are global properties of unfoldings, but many algorithms make local updates. A local core may look converged while the global approximation is still poor.

Main problems in adaptive TT methods

1. **Local error may not predict global error.** A local residual can be small for the current index sets but miss important unexplored regions.
2. **Bad sections can spoil interpolation.** In TT-cross, poor maxvol choices or unlucky samples can hide important directions.
3. **Ranks can grow too fast.** Over-enrichment increases memory and computation.
4. **Ranks can grow too slowly.** Under-enrichment causes stagnation.
5. **Optimization landscapes are nonconvex.** ALS-type methods can get stuck or converge slowly.

Two-site updates: DMRG and MALS idea

A one-site update optimizes one core at a time while ranks are fixed. A two-site update merges two neighboring cores, optimizes the merged object, and then splits it by SVD:

$$\mathcal{G}_k \mathcal{G}_{k+1} \longrightarrow \text{merged two-site block} \longrightarrow \text{SVD split with new rank.}$$

Because the SVD split can keep more singular vectors, the intermediate rank can increase. This is the basis of DMRG/MALS-style rank adaptation.

Enrichment and AMEn idea

AMEn, or alternating minimal energy with enrichment, adds carefully chosen directions to the local basis. These directions are often related to the residual or error equation. The goal is to enrich the search space with information about what the current approximation is missing.

Adaptive rank idea. Do not blindly enlarge ranks. Add directions that are likely to reduce the residual or improve unexplored parts of the tensor.

External error checks

For black-box TT-cross, an independent test set of entries can be used. The algorithm evaluates the approximation at random or structured validation multi-indices and estimates the relative error:

$$\frac{(\sum_{\omega \in \Omega} |x_{\omega} - y_{\omega}|^2)^{1/2}}{(\sum_{\omega \in \Omega} |x_{\omega}|^2)^{1/2}}.$$

This does not prove a global bound, but it is a practical stopping criterion.

A typical adaptive workflow

1. Start with small ranks.
2. Run fixed-rank sweeps or local optimization.
3. Estimate error by residual, validation samples, or stagnation measures.
4. If error is too large, enrich ranks by two-site splitting or residual directions.
5. Round the TT to remove unnecessary rank growth.
6. Repeat until the error, residual, or objective satisfies the target tolerance.

Tradeoff between stability and cost

Adaptive methods are a balancing act. More ranks give more expressive power but increase cost roughly polynomially in ranks. Too aggressive enrichment may make the method expensive; too conservative enrichment may cause stagnation. Good implementations combine enrichment, orthogonalization, truncation, and independent error checks.

What to reproduce in an exam

- Explain why ranks are often unknown and must be adapted.
- Describe problems with local error, poor sections, and rank overgrowth.
- Explain two-site updates and SVD-based rank increase.
- Explain enrichment and residual-based directions, especially AMEn-style thinking.
- Give practical stopping criteria.

Common pitfalls

- Adaptive rank search is not just “increase ranks until it works.” It needs error indicators and truncation.
- A small local residual is not always a reliable global certificate.
- Rank adaptation must be paired with orthogonalization and rounding for stability.

Reading map from the uploaded guide

- [OT10, pp. 80–88] for rank-adaptation issues in TT-cross.
- [HRS12, pp. A683–A713] for ALS/MALS optimization in TT format.
- [DS14, pp. A2248–A2271] for AMEn, enrichment, and rank adaptivity.
- [S14, pp. 230–244] for greedy and nested cross interpolation and error behavior.

9 Fast Elementwise Operations on TT Tensors Through a Special Version of TT-Cross

Question in plain language

How can we compute nonlinear elementwise functions of TT tensors without forming all entries or suffering huge rank growth?

The problem

Suppose $\mathcal{X}$ is in TT format and we want

$$y_{i_1 \dots i_d} = f(x_{i_1 \dots i_d}).$$

Examples include exponentials, logarithms, square roots, thresholds, reciprocals, or nonlinear combinations of two TT tensors:

$$y_{i_1 \dots i_d} = g(x_{i_1 \dots i_d}, z_{i_1 \dots i_d}).$$

The output tensor may have large TT ranks, and direct evaluation of all entries is impossible.

Formal arithmetic may be too expensive

Some elementwise operations have exact TT constructions. For example, the Hadamard product uses Kronecker products of slices and multiplies ranks. But nonlinear functions such as $\exp(\mathcal{X})$ or $1/(1 + \mathcal{X})$ do not have a simple low-rank exact construction in general.

Even for operations that do have exact constructions, ranks may explode. Therefore it can be better to build the result directly by approximation.

Special TT-cross viewpoint

Treat the output tensor $\mathcal{Y}$ as a black box. To evaluate one entry, do not form $\mathcal{Y}$. Instead:

1. Evaluate the needed entry or entries of the input TT tensors by contracting their cores at that multi-index.
2. Apply the scalar function f or g .
3. Return the scalar output value.

Thus the entry oracle is cheap:

$$(i_1, \dots, i_d) \mapsto f(x_{i_1 \dots i_d}).$$

Then TT-cross samples this oracle and constructs a compact TT approximation to $\mathcal{Y}$.

Key insight. We do not need a formula for the TT cores of $f(\mathcal{X})$. We only need a way to compute selected entries of $f(\mathcal{X})$. TT-cross turns those entries into a TT approximation.

Why rank adaptation is important

A nonlinear function can increase ranks unpredictably. Smooth functions applied to smooth or structured tensors may remain low-rank; discontinuous functions or sharp thresholds may require large ranks. Therefore fixed ranks are risky, and adaptive TT-cross is often used.

A practical method monitors validation error or stabilization of the approximation and increases ranks only where needed.

When this is preferable to explicit arithmetic

Special TT-cross for elementwise operations is preferable when:

- the full tensor is too large to form;
- entries of the output can be computed cheaply from input TT tensors;
- formal arithmetic would create very large intermediate ranks;
- an approximate result is acceptable;
- rank adaptation can detect the needed complexity.

Examples

Operation	TT-cross interpretation
$\mathcal{Y} = \exp(\mathcal{X})$	Evaluate x_i , return $\exp(x_i)$, cross-approximate the result.
$\mathcal{Y} = \mathcal{X} \odot \mathcal{Z}$	Either use exact Hadamard construction and round, or sample $x_i z_i$ directly if exact ranks are too large.
$\mathcal{Y} = 1/(1 + \mathcal{X}^2)$	Evaluate scalar nonlinear expression at sampled entries.
$\mathcal{Y} = \max(\mathcal{X}, 0)$	Use entrywise thresholding as a black-box function; expect ranks to grow if the threshold creates sharp boundaries.

What to reproduce in an exam

- Explain why direct elementwise computation is impossible for full high-dimensional tensors.
- Explain that the output is treated as a black-box tensor whose entries can be evaluated from input TT tensors.
- Explain why TT-cross and maxvol choose informative entries.
- Explain why rank adaptation is important for nonlinear operations.
- Compare this approach with explicit TT arithmetic followed by rounding.

Common pitfalls

- Do not assume nonlinear elementwise functions preserve TT ranks.
- Do not form all entries of $f(\mathcal{X})$.
- Do not skip validation: TT-cross gives an approximation, and nonlinear functions can create hidden complexity.

Reading map from the uploaded guide

- [O11, pp. 2308–2311] for elementwise/Hadamard products and rank growth in TT.
- [OT10, pp. 70–88] for TT-cross for black-box multidimensional arrays.
- [S14, pp. 217–244] for quasi-optimal maximum-volume cross interpolation for tensors.
- [DS14, pp. A2258–A2267] for enrichment and adaptivity ideas useful for nonlinear black-box outputs.

One-Page Exam Summary

Topic	Must remember
TT definition	Entry is a product of slices $\mathbf{G}_1(i_1) \cdots \mathbf{G}_d(i_d)$. Core k has size $r_{k-1} \times n_k \times r_k$, with $r_0 = r_d = 1$.
Storage	Exact count $\sum_k r_{k-1} n_k r_k$; for uniform sizes, about $d n r^2$.
TT unfoldings	$\mathbf{X}^{(k)}$ has rows $(i_1, \dots, i_k)$ and columns $(i_{k+1}, \dots, i_d)$.
Minimal ranks	$r_k = \text{rank}(\mathbf{X}^{(k)})$. This makes bounded TT-rank sets closed.
Arithmetic	Addition adds ranks; Hadamard product multiplies ranks; scalar products use sequential contractions. Round after rank-growing operations.
Orthogonality	Left/right core orthogonality is obtained by reshaping cores and applying QR/SVD sweeps. Mixed-canonical form stabilizes local computations.
TT-SVD	Sequential SVD of unfoldings. Local errors combine as $\sqrt{\sum_k \varepsilon_k^2}$. Quasi-optimality has factor $\sqrt{d-1}$.
TT-cross	Builds TT from selected entries using nested index sets and maxvol. Fixed-rank version assumes ranks are chosen.
Adaptive ranks	Use two-site updates, enrichment, AMEn-style residual directions, validation samples, and rounding.
Elementwise operations	Treat output as a black-box tensor: compute selected entries from input TT tensors, apply scalar operation, and run adaptive TT-cross.

Bibliography from the Uploaded Guide

The entries below are the references from the uploaded exam guide that are used most directly in this Tensor Train section.

- [DS14] Dolgov, S. V., Savostyanov, D. V. Alternating minimal energy methods for linear systems in higher dimensions. *SIAM Journal on Scientific Computing* 36(5), A2248–A2271, 2014.
- [DSL08] de Silva, V., Lim, L.-H. Tensor rank and the ill-posedness of the best low-rank approximation problem. *SIAM Journal on Matrix Analysis and Applications* 30(3), 1084–1127, 2008.
- [GOSTZ10] Goreinov, S. A., Oseledets, I. V., Savostyanov, D. V., Tyrtyshnikov, E. E., Zammarashkin, N. L. How to find a good submatrix. In *Matrix Methods: Theory, Algorithms and Applications*, World Scientific, 247–256, 2010.
- [GVL13] Golub, G. H., Van Loan, C. F. *Matrix Computations*. 4th edition. Johns Hopkins University Press, 2013.
- [HRS12] Holtz, S., Rohwedder, T., Schneider, R. The alternating linear scheme for tensor optimization in the tensor train format. *SIAM Journal on Scientific Computing* 34(2), A683–A713, 2012.
- [KB09] Kolda, T. G., Bader, B. W. Tensor decompositions and applications. *SIAM Review* 51(3), 455–500, 2009.
- [O11] Oseledets, I. V. Tensor-train decomposition. *SIAM Journal on Scientific Computing* 33(5), 2295–2317, 2011.
- [OT10] Oseledets, I. V., Tyrtyshnikov, E. E. TT-cross approximation for multidimensional arrays. *Linear Algebra and its Applications* 432(1), 70–88, 2010.

- [S14] Savostyanov, D. V. Quasioptimality of maximum-volume cross interpolation of tensors. *Linear Algebra and its Applications* 458, 217–244, 2014.
- [TB97] Trefethen, L. N., Bau, D. *Numerical Linear Algebra*. SIAM, 1997.
- [VVM12] Vannieuwenhoven, N., Vandebril, R., Meerbergen, K. A new truncation strategy for the higher-order singular value decomposition. *SIAM Journal on Scientific Computing* 34(2), A1027–A1052, 2012.